

Hands-on 4: Difference between JPA, Hibernate and Spring Data JPA

Scenario

This exercise explains the differences between JPA, Hibernate, and Spring Data JPA and demonstrates how data persistence works using both Hibernate and Spring Data JPA approaches.

Objective

The objective of this exercise is to understand:

- The role of JPA as a specification
 - Hibernate as an ORM implementation of JPA
 - Spring Data JPA as a higher-level abstraction over JPA
 - Reduction of boilerplate code using Spring Data JPA
-

1. Java Persistence API (JPA)

- JPA (Java Persistence API) is a **specification (JSR 338)** for managing relational data in Java applications.
 - It defines rules for ORM but does NOT provide implementation.
 - Frameworks like Hibernate implement JPA.
-

2. Hibernate

- Hibernate is a **powerful ORM tool** that implements JPA.
- It provides low-level database operations.
- Developers must manually manage:
 - Sessions
 - Transactions
 - Exception handling

Hibernate Code Example

```
public Integer addEmployee(Employee employee){  
    Session session = factory.openSession();  
    Transaction tx = null;  
    Integer employeeID = null;
```

```
try {  
    tx = session.beginTransaction();  
    employeeID = (Integer) session.save(employee);  
    tx.commit();  
} catch (HibernateException e) {  
    if (tx != null) tx.rollback();  
    e.printStackTrace();  
} finally {  
    session.close();  
}  
return employeeID;  
}
```

3. Spring Data JPA

- Spring Data JPA is an **abstraction over JPA implementations (like Hibernate)**.
 - It reduces boilerplate code significantly.
 - It automatically handles:
 - CRUD operations
 - Transactions
 - Repository implementation
-

Spring Data JPA Code Example

EmployeeRepository.java

```
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {  
}
```

EmployeeService.java

```
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.transaction.annotation.Transactional;
```

```
public class EmployeeService {
```

@Autowired

```
private EmployeeRepository employeeRepository;
```

@Transactional

```
public void addEmployee(Employee employee) {
```

```
    employeeRepository.save(employee);
```

```
}
```

```
}
```

4. Key Differences

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	ORM Framework	Abstraction Layer
Implementation	No	Yes	No (uses JPA provider)
Boilerplate Code	Medium	High	Low
Transaction Handling	No	Manual	Automatic
CRUD Operations	No	Yes	Yes (simplified)

5. Conclusion

JPA defines the standard for ORM in Java, Hibernate provides the actual implementation, and Spring Data JPA simplifies data access by reducing boilerplate code and automatically managing repositories and transactions. This layered architecture improves productivity and maintainability in enterprise applications.

6. References

- <https://dzone.com/articles/what-is-the-difference-between-hibernate-and-sprin-1>
- <https://www.javaworld.com/article/3379043/what-is-jpa-introduction-to-the-java-persistence-api.html>